SENTINEL-GNSS — Colab Training Notebook

Before running: Runtime ▸ Change runtime type ▸ **T4 GPU**

What lives where	
Code + all data CSVs	GitHub (cloned in Step 1)
Feature windows (.npz)	Built fresh each session from CSV
Checkpoints + figures	Google Drive (survives disconnects)

If Colab disconnects mid-training: Re-run Steps 1–3, then run the recovery cell at the bottom — it resumes from the last saved checkpoint automatically.

Step 1 — Clone repo from GitHub

All code and processed CSVs (including the labelled dataset) are pulled directly from GitHub.

```
import os; os.chdir('/content')
```

```
import os, shutil

GITHUB_REPO = 'https://github.com/Jorshuare/AI-Based-Prediction-for-GNSS-Signal-Degradation.git'
REPO_DIR    = '/content/sentinel-gnss'

# Always anchor to /content first so the kernel is never left in a deleted directory
%cd /content

if os.path.exists(f'{REPO_DIR}/.git'):
    print('Repo already cloned — pulling latest ...')
    %cd {REPO_DIR}
    !git pull
else:
    # Remove stale directory if it exists without .git (e.g. leftover from failed clone)
    if os.path.exists(REPO_DIR):
        print('Removing stale directory ...')
        shutil.rmtree(REPO_DIR)
    print('Cloning repo ...')
    !git clone {GITHUB_REPO} {REPO_DIR}
    %cd {REPO_DIR}

print(f'\nWorking directory: {os.getcwd()}')
!git log --oneline -5

# Confirm labelled CSV came down with the repo
csv_path = f'{REPO_DIR}/data/labelled/sentinel_gnss_labelled.csv'
assert os.path.exists(csv_path), f'CSV not found at {csv_path}'
import pandas as pd
df = pd.read_csv(csv_path)
print(f'\nDataset: {len(df):,} rows × {len(df.columns)} columns — ready.')
```

```
/content
Cloning repo ...
Cloning into '/content/sentinel-gnss'...
remote: Enumerating objects: 415, done.
remote: Counting objects: 100% (92/92), done.
remote: Compressing objects: 100% (72/72), done.
```

```
remote: Total 415 (delta 40), reused 65 (delta 19), pack-reused 323 (from 1)
Receiving objects: 100% (415/415), 264.34 MiB | 25.07 MiB/s, done.
Resolving deltas: 100% (119/119), done.
Updating files: 100% (231/231), done.
/content/sentinel-gnss

Working directory: /content/sentinel-gnss
c2d50f0 (HEAD -> main, origin/main, origin/HEAD) fix: balanced val subset for early stopping (500/class vs 97% CLEAN full val)
59c3212 fix: aux loss skipped for ablation models; baselines table handles flat metrics JSON
0aef1cd feat: aux t+0 head, warning weight [1,4,3], threshold tuning, drone cap, no-smote training
9531888 fix: train without smote"
de647d6 fix: softer class weights [1,2,3], remove titles, fix saliency backward, add __init__.py

Dataset: 97,393 rows × 41 columns — ready.
```

Step 2 — Install extra dependencies + verify GPU

Colab already ships with PyTorch, NumPy, pandas, scikit-learn, matplotlib, seaborn, scipy.
Only `imbalanced-learn` (SMOTE) needs installing.

```
!pip install -q imbalanced-learn

import torch
print(f'PyTorch  : {torch.__version__}')
print(f'CUDA     : {torch.cuda.is_available()}')
if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    print(f'GPU      : {torch.cuda.get_device_name(0)}')
    print(f'VRAM     : {props.total_memory / 1e9:.1f} GB')
else:
    print('WARNING: No GPU — go to Runtime > Change runtime type > T4 GPU')
```

```
PyTorch  : 2.10.0+cu128
CUDA     : True
GPU      : Tesla T4
VRAM     : 15.6 GB
```

Step 3 — Mount Google Drive (checkpoints + figures only)

Checkpoints are saved to Drive so training survives disconnects.
Data CSVs are **not** needed here — they came from GitHub in Step 1.

```
from google.colab import drive
import os, shutil

drive.mount('/content/drive')

REPO_DIR   = '/content/sentinel-gnss'
DRIVE_BASE = '/content/drive/MyDrive/sentinel-gnss'

# Folders that must persist across sessions
PAIRS = [
    (f'{REPO_DIR}/results/models/checkpoints', f'{DRIVE_BASE}/checkpoints'),
    (f'{REPO_DIR}/results/figures',            f'{DRIVE_BASE}/figures'),
]

for local, on_drive in PAIRS:
    os.makedirs(on_drive, exist_ok=True)
```

```
    os.makedirs(os.path.dirname(local), exist_ok=True)
    if os.path.isdir(local) and not os.path.islink(local):
        shutil.rmtree(local)   # remove empty local dir
    if not os.path.islink(local):
        os.symlink(on_drive, local)
    print(f'  {os.path.basename(local):15s} → {on_drive}')

# Report any existing checkpoints (from a previous run)
ckpt_dir = f'{DRIVE_BASE}/checkpoints'
ckpts = sorted(f for f in os.listdir(ckpt_dir) if f.endswith('.pt'))
if ckpts:
    print(f'\nExisting checkpoints on Drive ({len(ckpts)}):')
    for c in ckpts:
        print(f'  {c}')
    print('\nRun Step 5 with --resume to continue training.')
else:
    print('\nNo existing checkpoints — will start fresh.')
```

```
Mounted at /content/drive
  checkpoints     → /content/drive/MyDrive/sentinel-gnss/checkpoints
  figures         → /content/drive/MyDrive/sentinel-gnss/figures

No existing checkpoints — will start fresh.
```

Step 4 — Build feature windows

Reads `data/labelled/sentinel_gnss_labelled.csv` → produces sliding-window tensors.
Skips automatically if windows already exist (safe to re-run).

```
%cd /content/sentinel-gnss
# Build SMOTE windows (used by baselines) — --force ensures y_0s label is included
!python -m src.models.feature_prep --force

# Build no-SMOTE windows (used by deep model — SMOTE creates incoherent temporal windows)
!python -m src.models.feature_prep --no_smote --force

# Sanity check
import numpy as np
print('\nWindow shapes (SMOTE — for baselines):')
for split in ('train', 'val', 'test'):
    d = np.load(f'data/processed/windows/{split}.npz')
    y0_label = "y_0s present" if "y_0s" in d else "y_0s MISSING"
    print(f'  {split:5s}  X={d["X"].shape}  CLEAN={np.sum(d["y_5s"]==0):,}  WARNING={np.sum(d["y_5s"]==1):,}  DEGRADED={np.sum(d["y_5s"]==2):,}  [{y0_label}]')

print('\nWindow shapes (no-SMOTE — for deep model):')
for split in ('train', 'val', 'test'):
    d = np.load(f'data/processed/windows_no_smote/{split}.npz')
    y0_label = "y_0s present" if "y_0s" in d else "y_0s MISSING"
    print(f'  {split:5s}  X={d["X"].shape}  CLEAN={np.sum(d["y_5s"]==0):,}  WARNING={np.sum(d["y_5s"]==1):,}  DEGRADED={np.sum(d["y_5s"]==2):,}  [{y0_label}]')
```

```
/content/sentinel-gnss
INFO  Loading /content/sentinel-gnss/data/labelled/sentinel_gnss_labelled.csv …
INFO    97,393 rows × 41 columns
INFO    label dist: {0: 73370, 1: 15075, 2: 8948}
INFO    split dist: {'train': 53077, 'val': 26525, 'test': 17791}
INFO  Feature columns (34): ['alt', 'baseline_sats', 'clock_bias', 'cnr_trend', 'cnr_variance', 'cycle_slips', 'dop_ratio', 'elevation_violations', 'fix_continuity', 'fix_transitions', 'gdop', 'hdop', '
INFO    Scaler saved → /content/sentinel-gnss/data/processed/scaler.pkl
WARNING    Session 'scenario_a_r1' too short; skipping.
WARNING    Session 'scenario_a_r2' too short; skipping.
INFO    train: 52,757 windows  | 5s label dist: [35223 10600  6934]
INFO    val  : 23,842 windows  | 5s label dist: [23035   444   363]
```

```
INFO    test : 17,803 windows  | 5s label dist: [13740  3003  1060]
INFO  Applying SMOTE to training set …
INFO    SMOTE input  shape: (52757, 1020) | labels: [35223 10600  6934]
INFO    SMOTE output shape: (105669, 1020) | labels: [35223 35223 35223]
INFO    Saved train windows → /content/sentinel-gnss/data/processed/windows/train.npz  (105,669 samples)
INFO    Saved val   windows → /content/sentinel-gnss/data/processed/windows/val.npz  (23,842 samples)
INFO    Saved test  windows → /content/sentinel-gnss/data/processed/windows/test.npz  (17,803 samples)

=== Window summary ===
  train  X=(105669, 30, 34)  y_5s=[35223 35223 35223]
  val    X=(23842, 30, 34)  y_5s=[23035   444   363]
  test   X=(17803, 30, 34)  y_5s=[13740  3003  1060]
INFO  === --no_smote: saving class-weight-only windows to windows_no_smote/ ===
INFO  Loading /content/sentinel-gnss/data/labelled/sentinel_gnss_labelled.csv …
INFO    97,393 rows × 41 columns
INFO    label dist: {0: 73370, 1: 15075, 2: 8948}
INFO    split dist: {'train': 53077, 'val': 26525, 'test': 17791}
INFO  Feature columns (34): ['alt', 'baseline_sats', 'clock_bias', 'cnr_trend', 'cnr_variance', 'cycle_slips', 'dop_ratio', 'elevation_violations', 'fix_continuity', 'fix_transitions', 'gdop', 'hdop', '
INFO    Scaler saved → /content/sentinel-gnss/data/processed/scaler.pkl
WARNING    Session 'scenario_a_r1' too short; skipping.
WARNING    Session 'scenario_a_r2' too short; skipping.
INFO    train: 52,757 windows  | 5s label dist: [35223 10600  6934]
INFO    val  : 23,842 windows  | 5s label dist: [23035   444   363]
INFO    test : 17,803 windows  | 5s label dist: [13740  3003  1060]
INFO    Saved train windows → /content/sentinel-gnss/data/processed/windows_no_smote/train.npz  (52,757 samples)
INFO    Saved val   windows → /content/sentinel-gnss/data/processed/windows_no_smote/val.npz  (23,842 samples)
INFO    Saved test  windows → /content/sentinel-gnss/data/processed/windows_no_smote/test.npz  (17,803 samples)

=== Window summary ===
  train  X=(52757, 30, 34)  y_5s=[35223 10600  6934]
  val    X=(23842, 30, 34)  y_5s=[23035   444   363]
  test   X=(17803, 30, 34)  y_5s=[13740  3003  1060]

Window shapes (SMOTE — for baselines):
  train  X=(105669, 30, 34)  CLEAN=35,223  WARNING=35,223  DEGRADED=35,223  [y_0s present]
  val    X=(23842, 30, 34)  CLEAN=23,035  WARNING=444  DEGRADED=363  [y_0s present]
  test   X=(17803, 30, 34)  CLEAN=13,740  WARNING=3,003  DEGRADED=1,060  [y_0s present]

Window shapes (no-SMOTE — for deep model):
  train  X=(52757, 30, 34)  CLEAN=35,223  WARNING=10,600  DEGRADED=6,934  [y_0s present]
  val    X=(23842, 30, 34)  CLEAN=23,035  WARNING=444  DEGRADED=363  [y_0s present]
  test   X=(17803, 30, 34)  CLEAN=13,740  WARNING=3,003  DEGRADED=1,060  [y_0s present]
```

```
!pip install -q xgboost
!python -m src.models.baselines
```

```
INFO   Loading windows …
INFO     Train: (105669, 1020)   Test: (17803, 1020)
INFO
── Tier 1: Majority-class ──────────────────────────────────
INFO
── Tier 2: C/N₀ threshold rule ─────────────────────────────
INFO
── Tier 3: Random Forest ───────────────────────────────────
INFO     RandomForest  +5s  MacroF1=0.5443  MCC=0.4171
INFO     RandomForest  +15s  MacroF1=0.5327  MCC=0.4025
INFO     RandomForest  +30s  MacroF1=0.5183  MCC=0.3813
INFO
── Tier 3: XGBoost ─────────────────────────────────────────
INFO     XGBoost       +5s  MacroF1=0.5543  MCC=0.4296
INFO     XGBoost       +15s  MacroF1=0.5335  MCC=0.4055
INFO     XGBoost       +30s  MacroF1=0.5206  MCC=0.3902
INFO

INFO   ==============================================================================
INFO     Comparison Table — Test Set Macro-F1  (higher is better)
INFO   ==============================================================================
```

```
INFO    Method                      Justification                +5s     +15s     +30s
INFO  ------------------------------------------------------------------------------
INFO    MajorityClass               Trivial floor              0.2904   0.0377   0.0380
INFO    CNR_Threshold               Domain rule (RTCM)         0.0375   0.0377   0.0380
INFO    RandomForest                Classical ML baseline      0.5443   0.5327   0.5183
INFO    XGBoost                     Classical ML baseline      0.5543   0.5335   0.5206
INFO  ==============================================================================
INFO    Macro-F1: equally weights CLEAN / WARNING / DEGRADED classes.
INFO    Random 3-class predictor would score ≈ 0.3333.
INFO  ==============================================================================
INFO
Full metrics saved → /content/sentinel-gnss/results/baselines/baseline_comparison.json
```

Step 5 — Train

`--resume` continues from the last Drive checkpoint if it exists, otherwise starts fresh.
Checkpoints are saved directly to Drive every 10 epochs + whenever a new best val F1 is reached.

```
import shutil, os, glob

# Wipe ALL checkpoints from Drive so previous runs don't interfere
drive_ckpt = '/content/drive/MyDrive/sentinel-gnss/checkpoints'
local_ckpt = '/content/sentinel-gnss/results/models/checkpoints'

for ckpt_path in [drive_ckpt, local_ckpt]:
    if os.path.exists(ckpt_path):
        for f in glob.glob(f'{ckpt_path}/*'):
            os.remove(f)
        print(f'Cleared: {ckpt_path}')

%cd /content/sentinel-gnss
# Deep model trained WITHOUT SMOTE.
# Class weights [1.0, 4.0, 3.0] handle imbalance (WARNING boosted most — lowest precision).
# Auxiliary t+0 head forces model to learn current state as prerequisite for prediction.
!python -m src.models.train --batch_size 256 --window_dir data/processed/windows_no_smote
```

```
Cleared: /content/drive/MyDrive/sentinel-gnss/checkpoints
Cleared: /content/sentinel-gnss/results/models/checkpoints
/content/sentinel-gnss
INFO  Device: cuda
INFO    GPU: Tesla T4
INFO  Loading windows …
INFO    train batches: 207  val batches: 94  val_stop (balanced): 1089 samples
/content/sentinel-gnss/src/models/transformer_lstm.py:204: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True
  self.transformer = nn.TransformerEncoder(
SentinelGNSS  │  parameters: 367,692
INFO
============================================================
INFO    Training SENTINEL-GNSS  │  367,692 parameters
INFO    Epochs: 0 → 150  │  Patience: 20
INFO  ============================================================

INFO  Epoch 001/150  │ tr_loss=0.3899  val_loss=0.0729  │ val_F1(5s/15s/30s): 0.835/0.798/0.754  │ stop_F1=0.805  │ lr=4.00e-04  6.6s
INFO    ★ New best stop macro-F1 = 0.8046  → checkpoint_best.pt
INFO  Epoch 002/150  │ tr_loss=0.2288  val_loss=0.0613  │ val_F1(5s/15s/30s): 0.810/0.767/0.739  │ stop_F1=0.808  │ lr=6.00e-04  6.5s
INFO    ★ New best stop macro-F1 = 0.8078  → checkpoint_best.pt
INFO  Epoch 003/150  │ tr_loss=0.2017  val_loss=0.0585  │ val_F1(5s/15s/30s): 0.807/0.744/0.740  │ stop_F1=0.831  │ lr=8.00e-04  5.4s
INFO    ★ New best stop macro-F1 = 0.8310  → checkpoint_best.pt
INFO  Epoch 004/150  │ tr_loss=0.1806  val_loss=0.0564  │ val_F1(5s/15s/30s): 0.844/0.818/0.804  │ stop_F1=0.859  │ lr=1.00e-03  6.0s
INFO    ★ New best stop macro-F1 = 0.8590  → checkpoint_best.pt
INFO  Epoch 005/150  │ tr_loss=0.1676  val_loss=0.0549  │ val_F1(5s/15s/30s): 0.802/0.770/0.741  │ stop_F1=0.838  │ lr=1.00e-03  5.8s
INFO  Epoch 006/150  │ tr_loss=0.1561  val_loss=0.0595  │ val_F1(5s/15s/30s): 0.844/0.816/0.736  │ stop_F1=0.854  │ lr=1.00e-03  5.3s
INFO  Epoch 007/150  │ tr_loss=0.1484  val_loss=0.0591  │ val_F1(5s/15s/30s): 0.821/0.790/0.733  │ stop_F1=0.835  │ lr=1.00e-03  6.4s
```

```
INFO  Epoch 008/150  | tr_loss=0.1406  val_loss=0.0643  | val_F1(5s/15s/30s): 0.823/0.773/0.752  | stop_F1=0.830  | lr=9.99e-04  5.5s
INFO  Epoch 009/150  | tr_loss=0.1341  val_loss=0.0618  | val_F1(5s/15s/30s): 0.829/0.800/0.748  | stop_F1=0.848  | lr=9.98e-04  6.5s
INFO  Epoch 010/150  | tr_loss=0.1275  val_loss=0.0722  | val_F1(5s/15s/30s): 0.805/0.764/0.743  | stop_F1=0.840  | lr=9.97e-04  5.4s
INFO    → Periodic checkpoint: checkpoint_epoch_010.pt
INFO  Epoch 011/150  | tr_loss=0.1224  val_loss=0.0710  | val_F1(5s/15s/30s): 0.801/0.789/0.757  | stop_F1=0.854  | lr=9.96e-04  5.5s
INFO  Epoch 012/150  | tr_loss=0.1162  val_loss=0.0830  | val_F1(5s/15s/30s): 0.793/0.749/0.730  | stop_F1=0.818  | lr=9.94e-04  6.2s
INFO  Epoch 013/150  | tr_loss=0.1154  val_loss=0.0830  | val_F1(5s/15s/30s): 0.840/0.770/0.660  | stop_F1=0.820  | lr=9.93e-04  5.2s
INFO  Epoch 014/150  | tr_loss=0.1084  val_loss=0.0906  | val_F1(5s/15s/30s): 0.753/0.721/0.687  | stop_F1=0.820  | lr=9.91e-04  6.6s
INFO  Epoch 015/150  | tr_loss=0.1051  val_loss=0.0718  | val_F1(5s/15s/30s): 0.824/0.769/0.764  | stop_F1=0.842  | lr=9.88e-04  5.3s
INFO  Epoch 016/150  | tr_loss=0.1015  val_loss=0.0907  | val_F1(5s/15s/30s): 0.823/0.756/0.752  | stop_F1=0.845  | lr=9.86e-04  6.1s
INFO  Epoch 017/150  | tr_loss=0.0976  val_loss=0.0807  | val_F1(5s/15s/30s): 0.819/0.780/0.737  | stop_F1=0.840  | lr=9.83e-04  5.6s
INFO  Epoch 018/150  | tr_loss=0.0967  val_loss=0.0939  | val_F1(5s/15s/30s): 0.796/0.754/0.711  | stop_F1=0.819  | lr=9.80e-04  5.2s
INFO  Epoch 019/150  | tr_loss=0.0911  val_loss=0.0843  | val_F1(5s/15s/30s): 0.829/0.777/0.742  | stop_F1=0.840  | lr=9.77e-04  6.5s
INFO  Epoch 020/150  | tr_loss=0.0880  val_loss=0.0934  | val_F1(5s/15s/30s): 0.825/0.796/0.770  | stop_F1=0.838  | lr=9.74e-04  5.1s
INFO    → Periodic checkpoint: checkpoint_epoch_020.pt
INFO  Epoch 021/150  | tr_loss=0.0864  val_loss=0.0959  | val_F1(5s/15s/30s): 0.814/0.753/0.719  | stop_F1=0.835  | lr=9.70e-04  6.7s
INFO  Epoch 022/150  | tr_loss=0.0832  val_loss=0.1051  | val_F1(5s/15s/30s): 0.806/0.795/0.766  | stop_F1=0.835  | lr=9.66e-04  5.1s
INFO  Epoch 023/150  | tr_loss=0.0805  val_loss=0.1122  | val_F1(5s/15s/30s): 0.824/0.786/0.721  | stop_F1=0.820  | lr=9.62e-04  5.3s
INFO  Epoch 024/150  | tr_loss=0.0775  val_loss=0.1138  | val_F1(5s/15s/30s): 0.803/0.751/0.700  | stop_F1=0.822  | lr=9.58e-04  6.5s
INFO
  Early stopping triggered at epoch 24 (patience=20, no improvement for 20 epochs).
INFO
Training complete.  Best val macro-F1 = 0.8590
INFO    Loaded checkpoint: epoch=4  best_metric=0.8590
INFO    History saved → /content/sentinel-gnss/results/models/checkpoints/training_history.json
```

Step 6 — Evaluate

Loads `checkpoint_best.pt` from Drive, runs all 14 analyses, saves figures to Drive.

```
%cd /content/sentinel-gnss
# --tune_thresholds: sweeps WARNING/DEGRADED probability thresholds on val set
#   to maximise macro-F1. Free improvement — no retraining needed.
# --window_dir: val/test splits are identical to SMOTE dir, but keeps paths consistent.
!python -m src.models.evaluate --tune_thresholds --window_dir data/processed/windows_no_smote
```

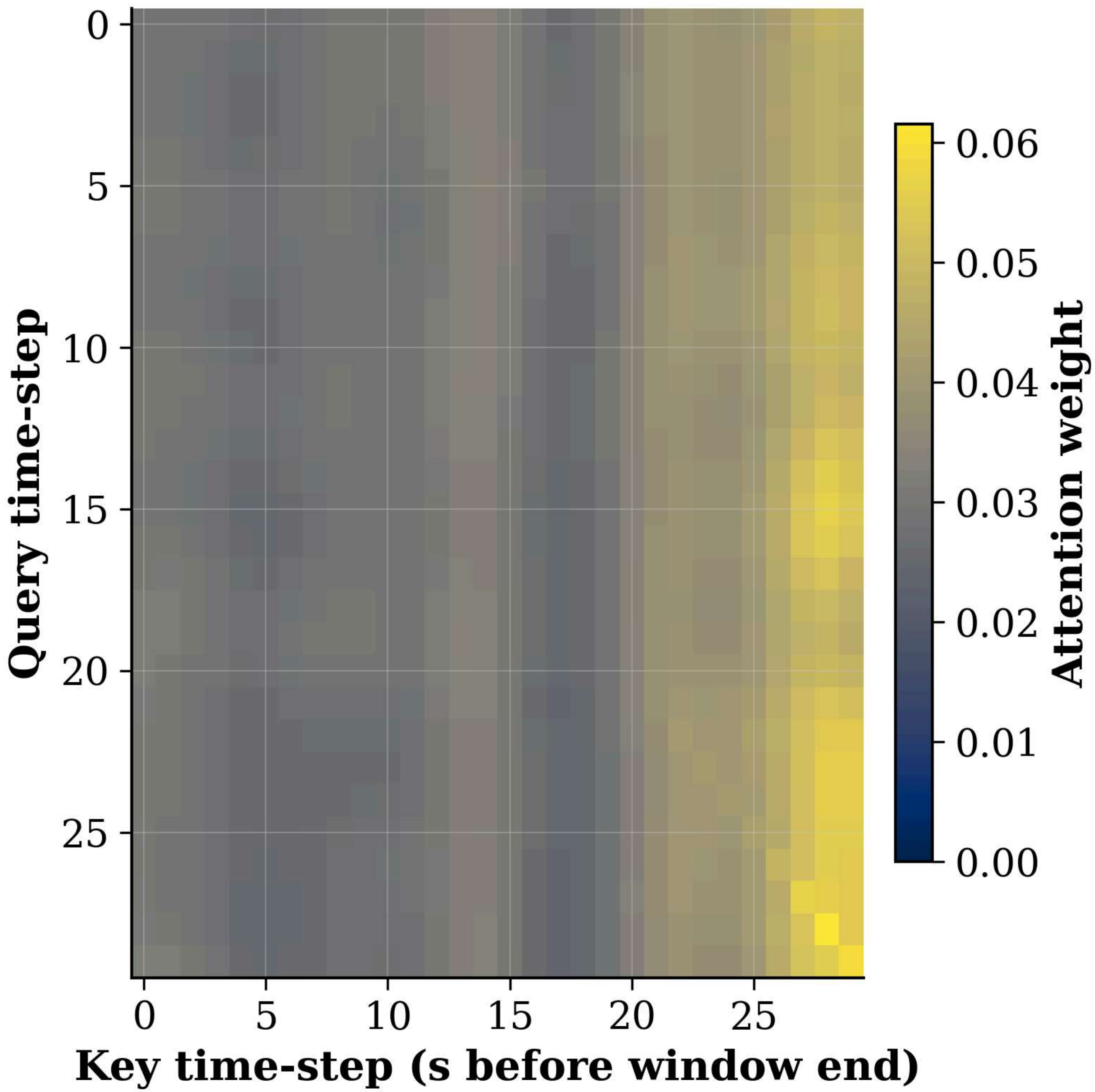
```
  Saved → /content/sentinel-gnss/results/figures/calibration_curves_test.png
INFO      ✓ Calibration curves
  Saved → /content/sentinel-gnss/results/figures/learning_curves.pdf
  Saved → /content/sentinel-gnss/results/figures/learning_curves.png
INFO      ✓ Learning curves
  Saved → /content/sentinel-gnss/results/figures/multi_horizon_comparison.pdf
  Saved → /content/sentinel-gnss/results/figures/multi_horizon_comparison.png
INFO      ✓ Multi-horizon comparison
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_clean.pdf
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_clean.png
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_warning.pdf
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_warning.png
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_degraded.pdf
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_degraded.png
INFO      ✓ Attention heatmaps
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_5s.pdf
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_5s.png
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_15s.pdf
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_15s.png
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_30s.pdf
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_30s.png
INFO      ✓ Feature saliency maps
  Saved → /content/sentinel-gnss/results/figures/lead_time_histogram.pdf
  Saved → /content/sentinel-gnss/results/figures/lead_time_histogram.png
INFO      ✓ Lead-time histogram
INFO
============================================================
INFO    All evaluation artefacts saved to: /content/sentinel-gnss/results/figures
INFO  ============================================================
```

Step 7 — View all figures inline

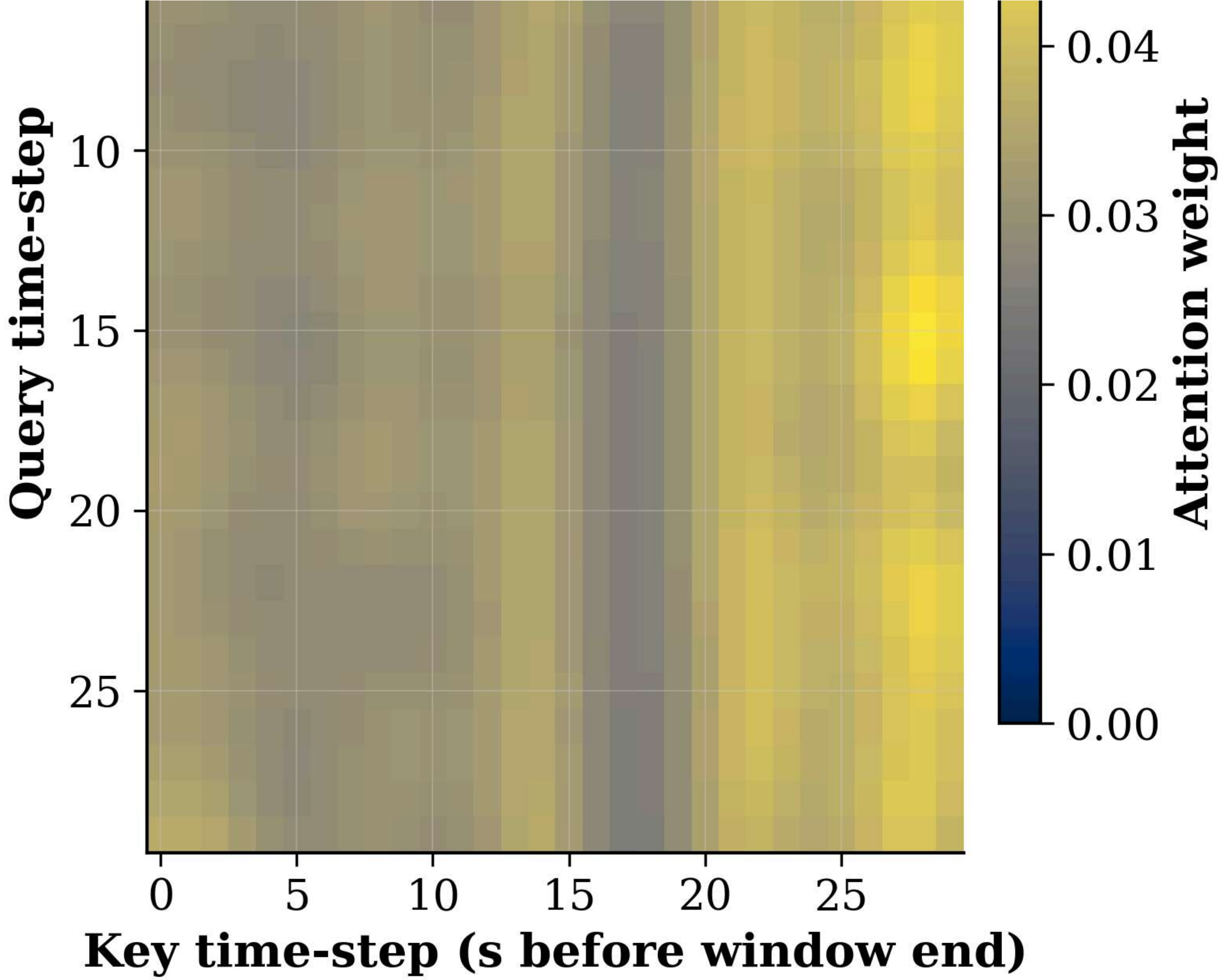
```
import glob
from IPython.display import Image, display

figs = sorted(glob.glob('/content/sentinel-gnss/results/figures/*.png'))
print(f'{len(figs)} figures saved to Drive:')
for f in figs:
    name = f.split('/')[-1]
    print(f'  {name}')
    display(Image(filename=f, width=950))
```

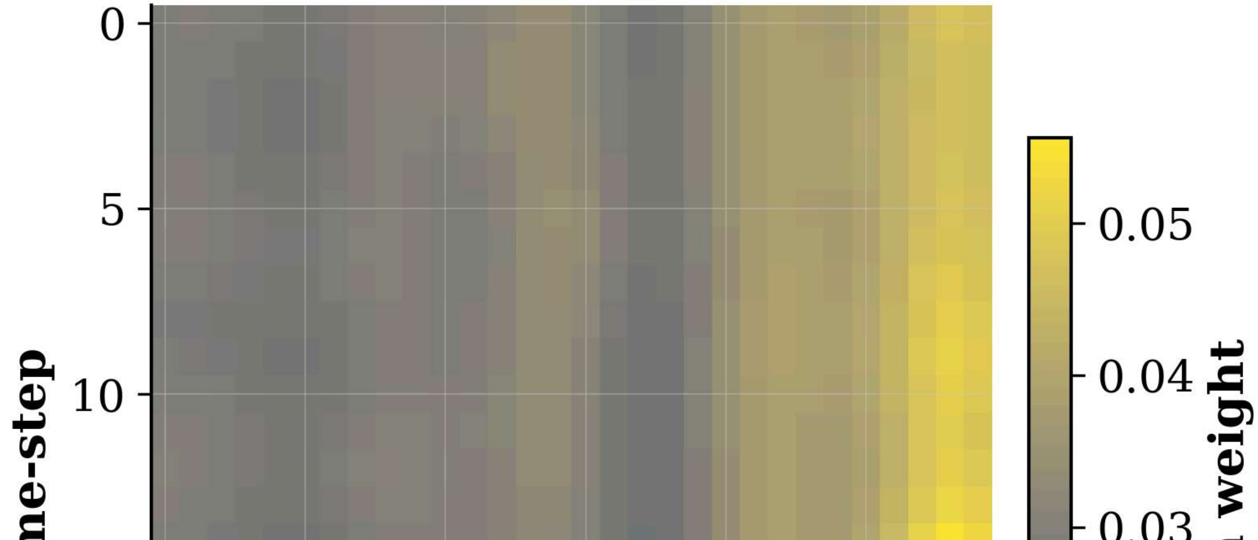
13 figures saved to Drive:
attention_heatmap_clean.png

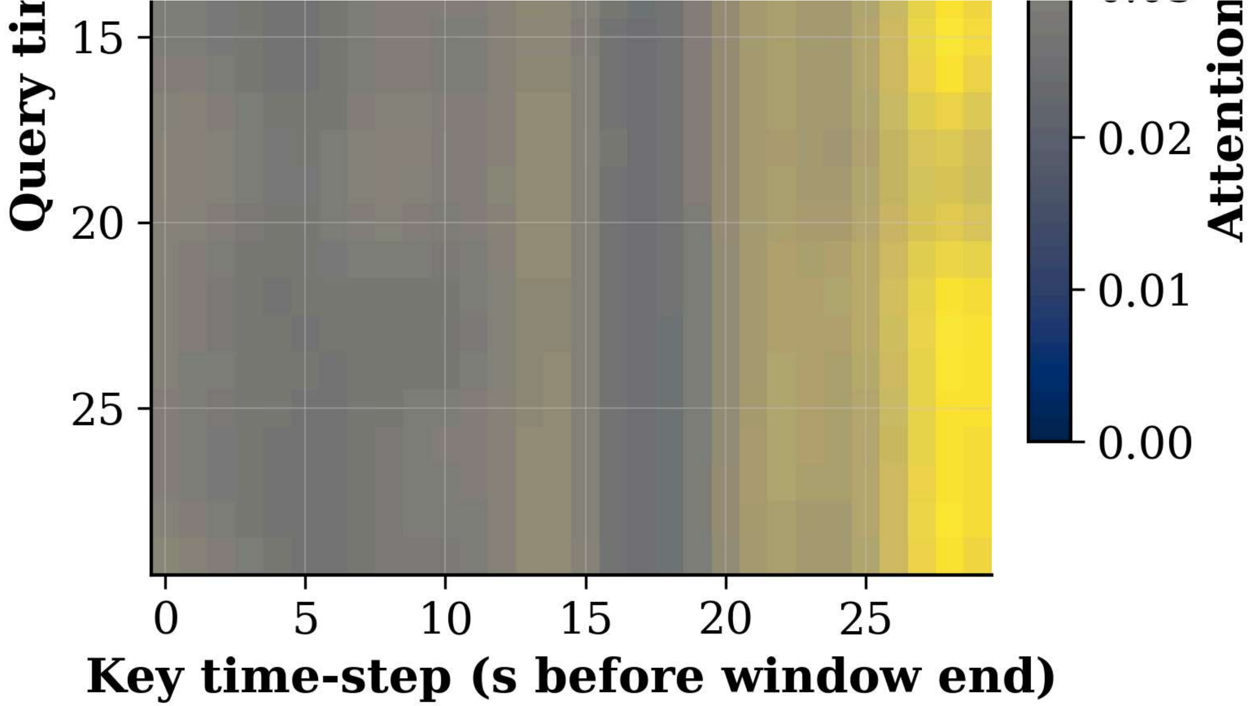


attention_heatmap_degraded.png

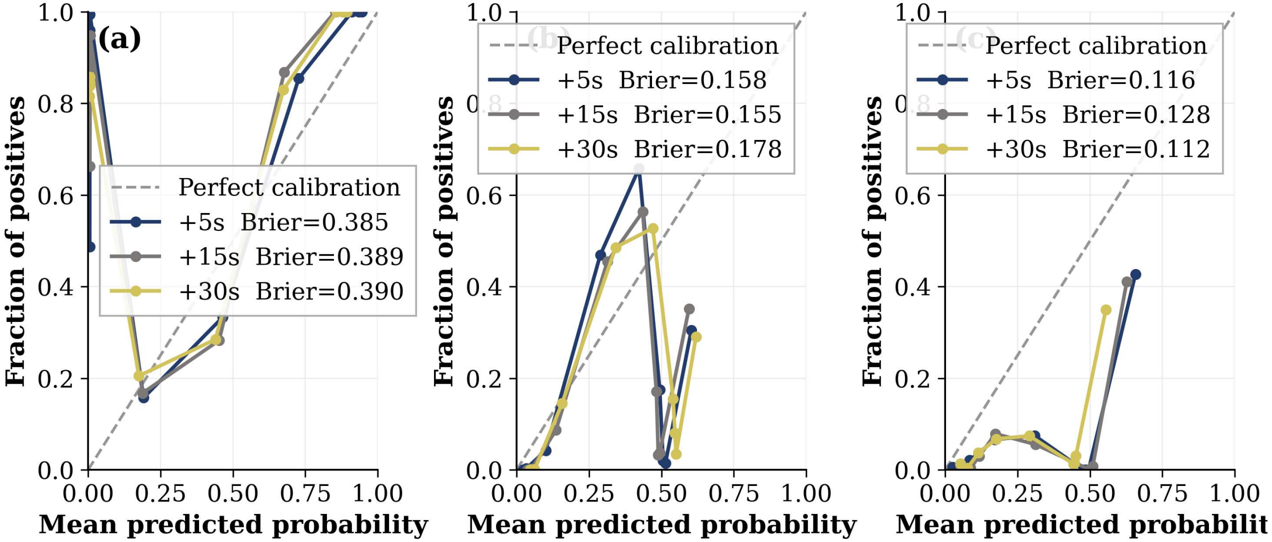


attention_heatmap_warning.png





calibration_curves_test.png



confusion_matrices_test.png